

グラフ理論 木 HLD（重軽分解） LCA（最小共通祖先）

重軽分解（HLD）

[解説動画はこちら](#)です。

1. 概要

本講座では、根付き木の **HLD（重軽分解）** について解説します。

HLD は根付き木の辺を **heavy edge**, **light edge** の 2 種に分けて扱うことで、木の頂点全体をいくつかのパスに分解するものです。

HLD は、木に対するさまざまな計算に幅広く使えるテクニックです。本講座では木の **パスクエリ**と呼ばれるタイプの問題を主に扱うこととして、それ以外の応用については別講座で扱うこととします。

2. 前提知識

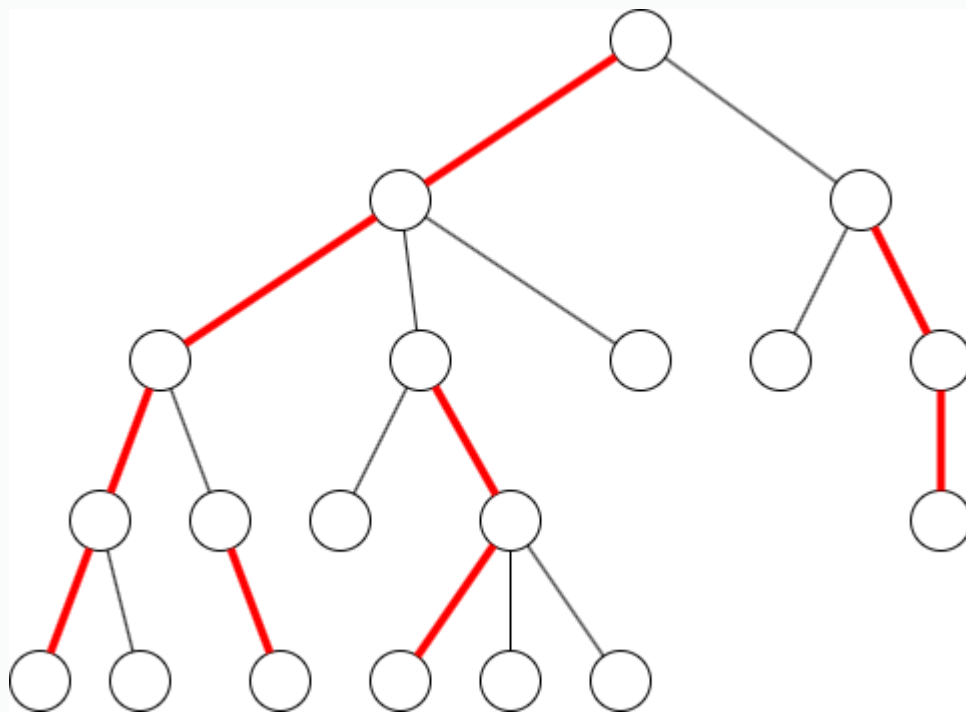
根付き木に関する用語。セグメント木などのデータ構造。

3. HLD

3.1. HLD とその性質

根付き木における子を持つ頂点について、部分木の頂点数が最も多い子をひとつとり、これを **heavy child**、それ以外の子を **light child** と呼ぶことにします。頂点と heavy child を結ぶ辺を **heavy edge**、頂点と light child を結ぶ辺を **light edge** といいます。

次の図では, heavy edge を赤太線, light edge を黒線で図示しています.



各頂点は親方向・子方向にそれぞれ 1 つ以下の heavy edge と接続することから, heavy edge 全体は頂点集合をいくつかのパスに分解します. この分解のこと, あるいは子や辺を heavy, light に分類することを, **重軽分解** (heavy-light decomposition) といいます. 以下, 重軽分解を **HLD** と表記することにします.

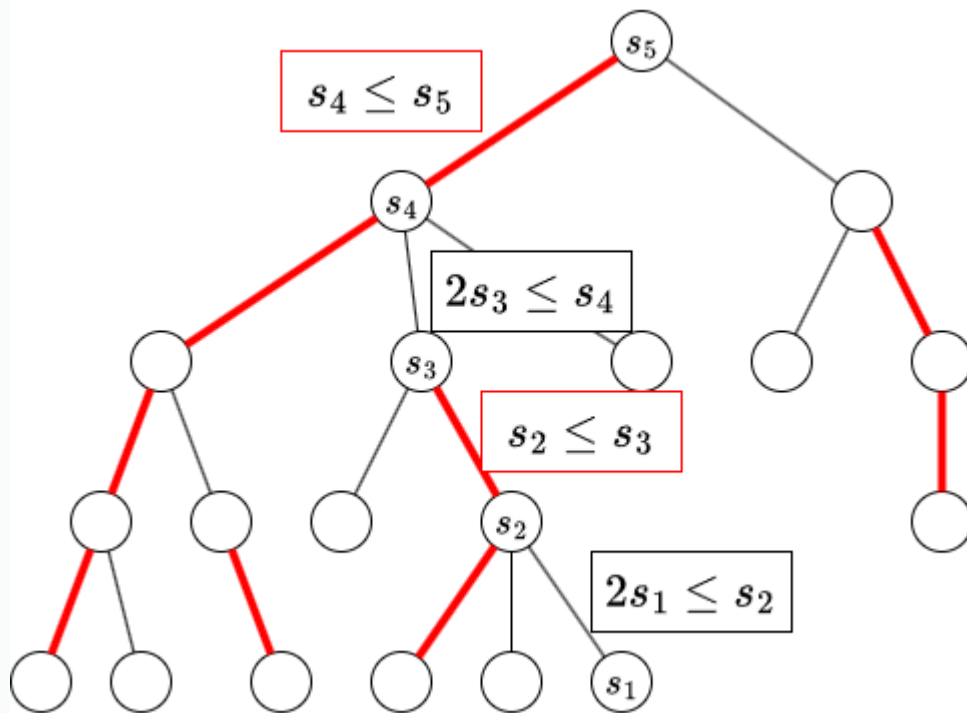
HLD の最も特徴的な性質は次の通りです。

【定理 1】

N 頂点の木の頂点 v に対して, v から根までのパスが通る light edge の個数は $\lfloor \log_2 N \rfloor$ 以下である.

v から根までのパスが通る頂点列を $v_1, v_2, \dots, v_k$ とします. s_i を v_i の部分木の頂点数とすれば, $1 \leq s_1 \leq s_2 \leq \dots \leq s_k \leq N$ が成り立ちます. よって, s_i が 2 倍以上に増加する i , つまり $s_{i+1} \geq 2s_i$ となる i の個数は $\lfloor \log_2 N \rfloor$ 以下です.

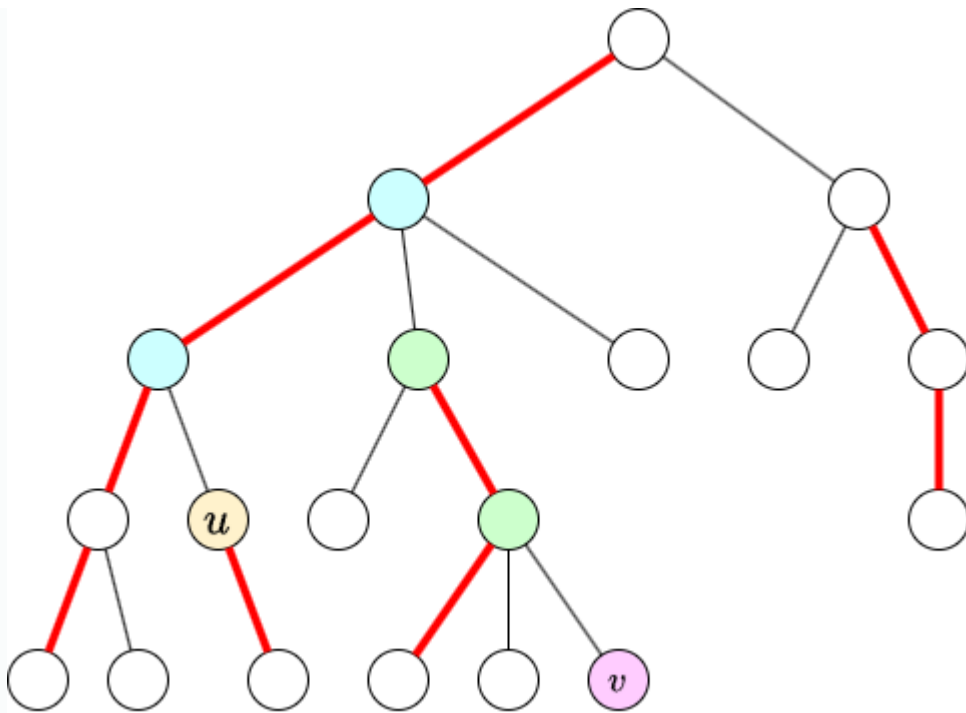
heavy child, light child の定義から, light edge を通る部分では s_i が 2 倍以上に増加するため, このパスは light edge を $\lceil \log_2 N \rceil$ 回以下しか通らないことが分かります. ■



【系 2】

N 頂点の木の頂点 u, v に対して, u から v までのパスが通る light edge の個数は $O(\log N)$ である. したがってこのパスは, 「頂点と heavy edge からなるパス」 および 「light edge」 $O(\log N)$ 個に分解できる.

パス $u \rightarrow v$ が通る辺は, u から根までのパス, v から根までのパスのどちらかに含まれるので, 【定理 1】 よりそのような light edge は $2 \lfloor \log_2 N \rfloor$ 以下であるため示されます. ■



3.2. 実装において保持するデータ

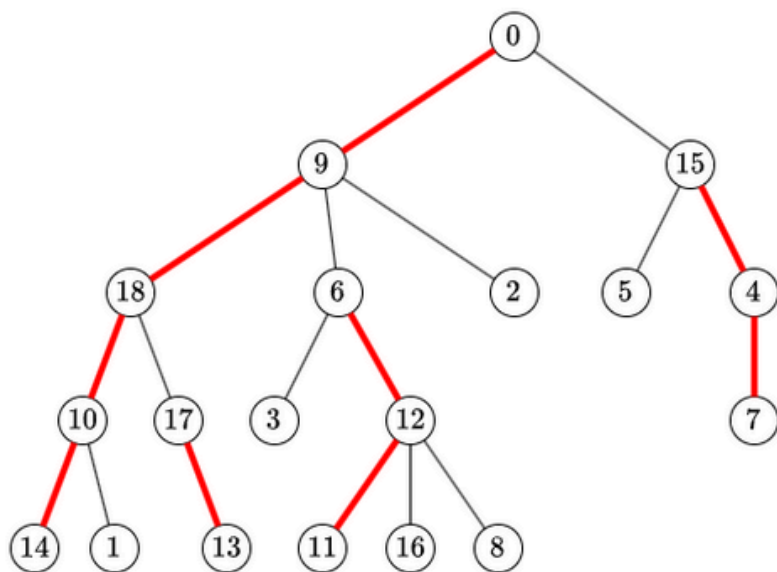
HLD の木の問題への応用や、パスクエリへの応用のためには、次のような要請があります。

- 各 heavy path 上の頂点列が分かる.
- 各頂点がどの heavy path の何番目にあるかを取得できる.
- 各 heavy path ごとの区間クエリを処理できる.

このような要請に応える簡潔な実装として現在広く用いられている実装方法について解説します。この実装方法には、部分木クエリも同時に処理できる強みもあります（部分木クエリについては、[オイラーツアーテクニック](#) を参考にしてください）。

次の配列を用意することを目指します。他に深さ、親などが必要になることも多いですが、ここでは省略しています。

- **heavy child 優先**で dfs したときに,
 - $\text{vertex}[i]$: 行きがけ順で i 番目に訪れる頂点 (ただし本講座では $i = 0$ から数えることにします).
 - $\text{id}[v]$: v の行きがけ順で何番目に訪れるか.
 - $\text{head}[v]$: v を通る heavy path の先頭 (最も根側にある頂点).



i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
vertex[i]	0	9	18	10	14	1	17	13	6	12	11	16	8	3	2	15	4	7	5

v	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
id[v]	0	5	14	13	16	18	8	17	12	1	3	10	9	7	4	15	11	6	2
head[v]	0	1	2	3	15	5	6	15	8	0	0	6	6	17	0	15	16	17	0

これらを得るためには、次のように 2 回の dfs を行えばよいです。

- 1 回目の dfs : 各頂点の部分木の頂点数を取得し, heavy child を求める.
- 2 回目の dfs : $\text{vertex}[i]$, $\text{id}[v]$, $\text{head}[v]$ などを計算する.

これらの計算量は $O(N)$ 時間です. より詳しい実装については 4.5 節の実装例や, [文献 2](#) を参照してください.

これらの配列を計算すると, 例えば次のような処理が可能になります.

- u, v が同じ heavy path に属するか否かを確認する : $\text{head}[u] = \text{head}[v]$ が成り立つかを確認すればよいです.
- v が heavy path の何番目の頂点であるかを確認する : $\text{id}[v] - \text{id}[\text{head}[v]]$ から求められます.
- v が属する heavy path の頂点を列挙する : $\text{head}[v]$ から行きがけ順に頂点を, 同じ heavy path に属する限り列挙します.

- heavy path 上の区間クエリ：heavy path 上の区間は頂点列 $\text{vertex}[0], \text{vertex}[1], \dots, \text{vertex}[N - 1]$ の区間でもあります。よって heavy path ごとにデータ構造を持つのではなく、頂点列 vertex に対するデータ構造をひとつ持って、この列に対する区間クエリを行えばよいです。（ただし、heavy path ごとにデータ構造を用意した方がパフォーマンスが良くなる場合もあります。）

3.3. 定義や用語に関する注意

HLD は、1983 年に [文献 1](#) で導入されました。この際には頂点 v に対して、部分木の頂点数が v の半分以上であるような子を heavy child とするという定義が用いられており、この定義は本講座で扱った定義とは異なります。この定義を用いる場合、

- 各頂点に対して heavy child は存在するならば一意であること。
- heavy path において最も深い頂点が葉とは限らないこと。

などの違いが生じます。ただしどちらの定義を用いても、[【定理 1】](#) や [【系 2】](#) は成り立ち、本講座内のその他の部分についてもほとんど影響がありません。

他にも葉の個数が最大であるような部分木への辺を heavy child とする文献なども見られ、文献によって HLD の定義に細かな差が見られるので注意してください。

4. レベル祖先クエリと LCA クエリ

ある頂点から親方向に移動して根に到達する過程が、heavy path 上での移動と light edge による移動の合計 $O(\log N)$ 回になることで、いくつかの木に関する基礎的な計算を $O(\log N)$ 時間で行うことができます。本講座では、レベル祖先クエリおよび LCA クエリについて解説します。

4.1. レベル祖先クエリ

【問題 3】（レベル祖先クエリ）

N 頂点の根付き木が与えられます。 Q 個のクエリに答えてください。

クエリでは頂点 v および、 v の深さ以下の非負整数 d が与えられます。 v の祖先であって深さ d にあるものを求めてください。

上述の通りの HLD のデータを $O(N)$ 時間で計算しておき、各頂点の深さ $\text{depth}[v]$ や親 $\text{parent}[v]$ も計算しておきます。すると、クエリは次を繰り返すことで答えることができます。

- $\text{depth}[\text{head}[v]] \leq d$ であるならば、 v が属する heavy path の適切な位置にある頂点を出力し、終了する。
- そうでないならば、 v を $\text{parent}[\text{head}[v]]$ で置き換える。

後者のステップが行われる回数は、 v から根までの light edge の個数で上から抑えられるため、このアルゴリズムの計算量は $O(\log N)$ 時間です。

4.2. LCA クエリ

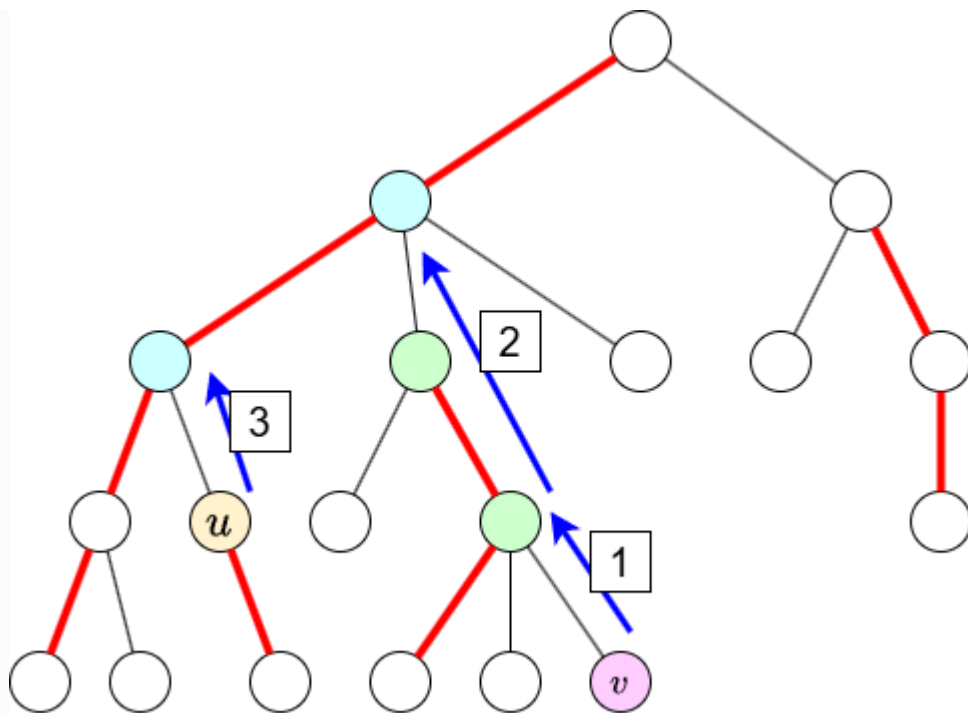
【問題 4】(LCA クエリ)

N 頂点の根付き木が与えられます。 Q 個のクエリに答えてください。

クエリでは 2 頂点 u, v が与えられるので、その LCA（最小共通祖先）を求めてください。つまり、 u から v へのパスが通る、深さ最小の頂点を求めてください。

2 頂点を親方向に移動させていくという考え方でアルゴリズムを構築できます。

- u, v が異なる heavy path 上にある場合
 - u, v の行きがけ順を比較し、順序が後である方を選ぶ。ここでは u が選ばれたとする。
 - u を $\text{head}[u]$ の親に置き換える。
- u, v が同じ heavy path 上にある場合
 - この場合には u, v は祖先・子孫の関係になっている。これらのうち深さが小さい方を出力する。



このアルゴリズムの計算量について考えます。 u を移動させる操作のたびに、 u は light edge を通ります。 HLD において、 頂点から根までのパスに含まれる light edge は $O(\log N)$ 個なので、 u を移動させる操作は $O(\log N)$ 回しか行われません。

同様に v を移動させる操作も $O(\log N)$ 回しか行われないため、 u, v の LCA は $O(\log N)$ 時間で計算できます。

4.3. 2 頂点の距離

LCA クエリを利用すると、 次の問題に答えることができます。

【問題 5】 (2 頂点の距離)

N 頂点の木が与えられます。 Q 個のクエリに答えてください。

クエリでは 2 頂点 u, v が与えられるので、 木における u, v の距離を求めてください。

適当に根を選んで根付き木にした上で、 $w = \text{LCA}(u, v)$ とします。 uv パスを、 $u \rightarrow w \rightarrow v$ と分割することで、 u, v の距離は

$$(\text{depth}[u] - \text{depth}[w]) + (\text{depth}[v] - \text{depth}[w])$$

であることが分かります.

4.4. Jump on Tree

レベル祖先クエリと LCA クエリを組み合わせると, 次の問題に答えることができます.

【問題 6】(Jump on Tree)

N 頂点の木が与えられます. Q 個のクエリに答えてください.

クエリでは 2 頂点 s, t および非負整数 i が与えられるので, 木上のパス st における i 番目の頂点を求めてください.

求める点は s, t のいずれかのレベル祖先であり, それがどちらの, どの深さの祖先になるかは $\text{LCA}(s, t)$ を求めて深さを調べれば分かります.

4.5. 実装例

Library Checker [Jump on Tree](#) への提出を紹介します. 実装には HLD を利用したレベル祖先クエリ, LCA クエリが含まれます.

- <https://judge.yosupo.jp/submission/327921>

5. パスクエリ

頂点を適当な順番に並べて列にしておくと, 木のパスは $O(\log N)$ 個の区間に分解できるのでした. このことから, 木のパスに対するさまざまなクエリが, 区間クエリ $O(\log N)$ 回により処理できます.

例えば以下のような問題を解くことができます.

【問題 7】

N 頂点の木が与えられます. 頂点 v は重み a_v を持ちます. 次の形式の Q クエリを処理してください.

- 1 $v\ x: a_v$ を x に変更.
- 2 $u\ v: u$ から v へのパスが通る頂点重みの最小値を出力.

セグメント木を用いて、構築 $O(N)$ 時間、タイプ 1 のクエリは $O(\log N)$ 時間、タイプ 2 のクエリは $O(\log^2 N)$ 時間で行えます。タイプ 2 のクエリの時間計算量は、 $O(\log N)$ 個の区間に対してセグメント木の区間クエリを行うことに由来します。

もしタイプ 1 のクエリがない場合には、スパーステーブルを用いて構築 $O(N \log N)$ 時間、タイプ 2 のクエリを $O(\log N)$ 時間で行うこともできます。

【問題 8】

N 頂点の木が与えられます。頂点 v は重み a_v を持ちます。次の形式の Q クエリを処理してください。

- 1 $v\ x: a_v$ を x に変更.
- 2 $u\ v: u$ から v へのパスが通る頂点重みの最小値を出力.
- 3 $u\ v\ x: u$ から v へのパスが通る頂点の重みに x を加算する.

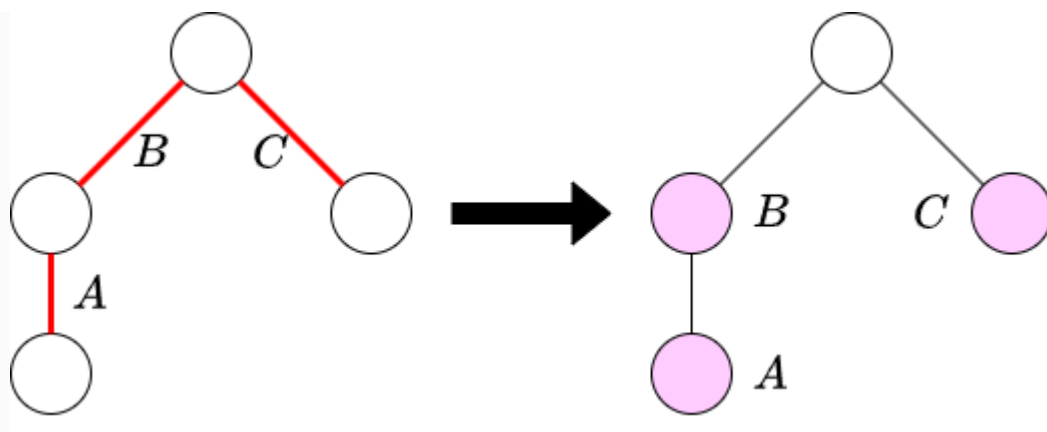
【問題 7】に比べてタイプ 3 のクエリが増えたとします。この場合には遅延セグメント木を利用すれば、やはりタイプ 1 のクエリを $O(\log N)$ 時間、タイプ 2, 3 のクエリを $O(\log^2 N)$ 時間で行えます。

【問題 9】

N 頂点の木が与えられます。辺 i は重み a_i を持ちます。次の形式の Q クエリを処理してください。

- 1 $i\ x: a_i$ を x に変更.
- 2 $u\ v: u$ から v へのパスが通る辺重みの最小値を出力.

パス上の**辺の列**に対するクエリです。辺の値をその辺の**子側の頂点**で保持することにすれば、頂点重みの場合のパスクエリとほとんど同じことになります。LCA を除く必要があることに注意してください。



【問題 10】 (Vertex Set Path Composite)

N 頂点の木が与えられます. 頂点 i は一次関数 $f_i(x) = a_i x + b_i$ を持ちます. 次の形式の Q クエリを処理してください.

- 1 $i \ a \ b$: a_i を a に変更し, b_i を b に変更する.
- 2 $u \ v \ x$: u から v へのパスが通る頂点を順に $v_1, v_2, \dots, v_k$ とするとき, $f_{v_k}(\dots f_{v_2}(f_{v_1}(x)) \dots)$ を出力する.

関数合成のように頂点进行处理する順番によって結果が変わる場合には, 列の左から右, 右から左どちらの方向にも計算できるように何らかの対処をすることが必要になります.

例えばセグメント木のノードごとのデータとして, 区間内の関数を左から順に合成したものと, 右から順に合成したものの両方を持たせることでこの問題は $O(N + Q \log^2 N)$ 時間で解くことができます.

5.1. 実装例

【問題 10】 として取り上げた Library Checker Vertex Set Path Composite の実装例を示します.

- <https://judge.yosupo.jp/submission/330097>

6. 関連問題

1. <https://judge.yosupo.jp/problem/lca>

2. <https://cses.fi/problemset/task/1135>
3. https://atcoder.jp/contests/abc014/tasks/abc014_4
4. https://judge.yosupo.jp/problem/jump_on_tree
5. <https://cses.fi/problemset/task/2134>
6. https://judge.yosupo.jp/problem/vertex_add_path_sum
7. https://judge.yosupo.jp/problem/vertex_set_path_composite
8. https://atcoder.jp/contests/abc294/tasks/abc294_g
9. <https://yukicoder.me/problems/no/399>
10. <https://yukicoder.me/problems/no/650>
11. https://atcoder.jp/contests/joisc2018/tasks/joisc2018_a

▶ 問題について

7. まとめ

本講座では、HLD（重軽分解）の定義，代表的な実装方法，パスクエリへの応用を解説しました。オイラーツアーテクニック による部分木クエリ処理と合わせると，かなり幅広い木クエリを処理できると思います。

HLD は，木に対するテクニックの中でも最も汎用的なもののひとつです。本講座で述べたものの以外にも重要な応用が複数あるため，AtCoder Algorithm Lectures でも改めて扱う予定です。

8. 参考文献

1. Daniel D. Sleator, Robert E. Tarjan. A Data Structure for Dynamic Trees. Journal of Computer and System Sciences. Vol. 26. No. 3. pp. 362–391. 1983. DOI: 10.1016/0022-0000(83)90006-5. [https://doi.org/10.1016/0022-0000\(83\)90006-5](https://doi.org/10.1016/0022-0000(83)90006-5). (閲覧日: 2026-03-31).
2. adamant. Easiest HLD with subtree queries. Codeforces Blog. <https://codeforces.com/blog/entry/53170>. (閲覧日: 2026-03-31).
3. cp-algorithms. Heavy-light decomposition. <https://cp-algorithms.com/graph/hld.html>. (閲覧日: 2026-03-31).

4. USACO Guide. Heavy-Light Decomposition. <https://usaco.guide/plt/hld>. (閲覧日: 2026-03-31).
5. Nyaan. Ex - Antichain 解説 (重軽分解の項). AtCoder Editorial. <https://atcoder.jp/contests/abc269/editorial/4838>. (閲覧日: 2026-03-31).
6. Pro_ktmr. 【図解】木のパスに関するクエリは HL 分解！ その仕組みと実装を図で理解する | Heavy-Light Decomposition. https://qiita.com/Pro_ktmr/items/4e1e051ea0561772afa3. (閲覧日: 2026-03-31).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

各サービス

[AtCoderとは？](#)

[AtCoder](#) ↗

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)